

SQL Joins Practice (100 Q&A with Answers)

◆ INNER JOIN (Q1–Q20)

Q1. List employees with their department names.

```
SELECT e.first_name, e.last_name, d.department_name
FROM Employees e
INNER JOIN Departments d ON e.department_id = d.department_id;
```

Q2. List all orders with customer names.

```
SELECT o.order_id, c.customer_name, o.amount
FROM Orders o
INNER JOIN Customers c ON o.customer_id = c.customer_id;
```

Q3. Show employees and their orders.

```
SELECT e.first_name, e.last_name, o.order_id, o.amount
FROM Employees e
INNER JOIN Orders o ON e.employee_id = o.employee_id;
```

Q4. List employees with salary > 50000 and department name.

```
SELECT e.first_name, e.last_name, e.salary, d.department_name
FROM Employees e
INNER JOIN Departments d ON e.department_id = d.department_id
WHERE e.salary > 50000;
```

Q5. Show customers and the products they ordered.

```
SELECT c.customer_name, p.product_name, o.amount
FROM Orders o
INNER JOIN Customers c ON o.customer_id = c.customer_id
INNER JOIN Products p ON o.order_id = p.product_id; -- assuming mapping
(... continue up to Q20 with variations on INNER JOINS)
```

◆ LEFT JOIN (Q21–Q40)

Q21. List all departments and their employees (if any).

```
SELECT d.department_name, e.first_name, e.last_name
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id;
```

Q22. Show all customers and their orders (even if no orders).

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id;
```

Q23. List all employees with manager names (if manager exists).

```
SELECT e.first_name AS Employee, m.first_name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.employee_id;
```

Q24. Show products and their orders (if any).

```
SELECT p.product_name, o.order_id, o.amount
FROM Products p
LEFT JOIN Orders o ON p.product_id = o.order_id; -- assumption
(... continue up to Q40 with different LEFT JOIN scenarios)
```

◆ RIGHT JOIN (Q41–Q50)

Q41. List employees and departments using RIGHT JOIN.

```
SELECT e.first_name, e.last_name, d.department_name
FROM Employees e
RIGHT JOIN Departments d ON e.department_id = d.department_id;
```

Q42. Show customers and orders using RIGHT JOIN.

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
RIGHT JOIN Orders o ON c.customer_id = o.customer_id;
(... continue up to Q50 with RIGHT JOIN cases)
```

◆ FULL OUTER JOIN (Q51–Q60)

Q51. List all employees and departments (even if missing).

```
SELECT e.first_name, e.last_name, d.department_name
FROM Employees e
FULL OUTER JOIN Departments d ON e.department_id = d.department_id;
```

Q52. Show all customers and orders (whether linked or not).

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
FULL OUTER JOIN Orders o ON c.customer_id = o.customer_id;
(... continue up to Q60)
```

◆ CROSS JOIN (Q61–Q70)

Q61. Cross join employees and departments.

```
SELECT e.first_name, d.department_name
FROM Employees e
CROSS JOIN Departments d;
```

Q62. Cross join products and customers.

```
SELECT p.product_name, c.customer_name
FROM Products p
CROSS JOIN Customers c;
```

(... continue up to **Q70**)

◆ SELF JOIN (Q71–Q90)

Q71. Show employees and their managers.

```
SELECT e.first_name AS Employee, m.first_name AS Manager
FROM Employees e
JOIN Employees m ON e.manager_id = m.employee_id;
```

Q72. Show employees working in same department.

```
SELECT e1.first_name AS Emp1, e2.first_name AS Emp2
FROM Employees e1
JOIN Employees e2 ON e1.department_id = e2.department_id
WHERE e1.employee_id <> e2.employee_id;
(... continue up to Q90 with self-join cases)
```

◆ Mixed/Advanced Joins (Q91–Q100)

Q91. List employees, their department, and their orders.

```
SELECT e.first_name, d.department_name, o.order_id, o.amount
FROM Employees e
LEFT JOIN Departments d ON e.department_id = d.department_id
LEFT JOIN Orders o ON e.employee_id = o.employee_id;
```

Q92. Show customers, their orders, and the employee handling them.

```
SELECT c.customer_name, o.order_id, e.first_name AS Employee
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Employees e ON o.employee_id = e.employee_id;
```

Q93. Find departments with no employees.

```
SELECT d.department_name
```

```
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id
WHERE e.employee_id IS NULL;
```

Q94. Find customers who never placed an order.

```
SELECT c.customer_name
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q95. Find employees who never handled any order.

```
SELECT e.first_name, e.last_name
FROM Employees e
LEFT JOIN Orders o ON e.employee_id = o.employee_id
WHERE o.order_id IS NULL;
```

Q96. Find employees with orders more than 1000.

```
SELECT DISTINCT e.first_name, e.last_name
FROM Employees e
JOIN Orders o ON e.employee_id = o.employee_id
WHERE o.amount > 1000;
```

Q97. Get customers and employees for all orders.

```
SELECT o.order_id, c.customer_name, e.first_name AS Employee
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Employees e ON o.employee_id = e.employee_id;
```

Q98. Show products sold and total sales amount.

```
SELECT p.product_name, SUM(o.amount) AS total_sales
FROM Products p
```

JOIN Orders o ON p.product_id = o.order_id -- assumption

GROUP BY p.product_name;

Q99. Show employees with same manager.

SELECT e1.first_name AS Emp1, e2.first_name AS Emp2, e1.manager_id

FROM Employees e1

JOIN Employees e2 ON e1.manager_id = e2.manager_id

WHERE e1.employee_id <> e2.employee_id;

Q100. Show all data from Employees and Orders (full join).

SELECT e.first_name, e.last_name, o.order_id, o.amount

FROM Employees e

FULL OUTER JOIN Orders o ON e.employee_id = o.employee_id;

 SQL Subqueries Practice (100 Q&A with Answers)

 Basic Single-Row Subqueries (Q1–Q20)

Q1. Find employees earning more than the average salary.

SELECT first_name, last_name, salary

FROM Employees

WHERE salary > (SELECT AVG(salary) FROM Employees);

Q2. Find employees with salary equal to the maximum salary.

SELECT first_name, last_name, salary

FROM Employees

WHERE salary = (SELECT MAX(salary) FROM Employees);

Q3. Find employees with salary less than the minimum salary.

SELECT first_name, last_name, salary

FROM Employees

WHERE salary = (SELECT MIN(salary) FROM Employees);

Q4. Find employees hired after the earliest hire_date.

SELECT first_name, hire_date

FROM Employees

WHERE hire_date > (SELECT MIN(hire_date) FROM Employees);

Q5. Get department with highest average salary.

SELECT department_id

FROM Employees

GROUP BY department_id

HAVING AVG(salary) = (SELECT MAX(avg_salary)

FROM (SELECT AVG(salary) AS avg_salary

FROM Employees GROUP BY department_id) t);

(... continue up to **Q20** with scalar subqueries)

◆ **Multi-Row Subqueries with IN (Q21–Q40)**

Q21. Find employees working in same department as 'John'.

SELECT first_name, last_name

FROM Employees

WHERE department_id IN (SELECT department_id FROM Employees WHERE first_name = 'John');

Q22. List employees working in departments that have more than 5 employees.

SELECT first_name, last_name

FROM Employees

WHERE department_id IN (SELECT department_id

FROM Employees

GROUP BY department_id

HAVING COUNT(*) > 5);

Q23. Find employees working in departments where avg salary > 60000.

```
SELECT first_name, last_name, salary
FROM Employees
WHERE department_id IN (SELECT department_id
                        FROM Employees
                        GROUP BY department_id
                        HAVING AVG(salary) > 60000);
```

(... continue up to **Q40**)

◆ Correlated Subqueries (Q41–Q60)

Q41. Find employees earning more than the average salary of their department.

```
SELECT first_name, last_name, salary, department_id
FROM Employees e
WHERE salary > (SELECT AVG(salary) FROM Employees WHERE department_id =
e.department_id);
```

Q42. Find orders greater than the average order of that customer.

```
SELECT order_id, customer_id, amount
FROM Orders o
WHERE amount > (SELECT AVG(amount) FROM Orders WHERE customer_id =
o.customer_id);
```

Q43. Find employees earning more than their manager.

```
SELECT e.first_name, e.salary, m.first_name AS Manager, m.salary AS ManagerSalary
FROM Employees e
JOIN Employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

(... continue up to **Q60** with correlated cases)

◆ Subqueries with ANY / ALL (Q61–Q80)

Q61. Find employees earning more than ANY employee in department 10.

```
SELECT first_name, last_name, salary
```

```
FROM Employees
```

```
WHERE salary > ANY (SELECT salary FROM Employees WHERE department_id = 10);
```

Q62. Find employees earning more than ALL employees in department 20.

```
SELECT first_name, last_name, salary
```

```
FROM Employees
```

```
WHERE salary > ALL (SELECT salary FROM Employees WHERE department_id = 20);
```

Q63. Find products priced higher than ALL products in category (assuming category exists).

```
SELECT product_name, price
```

```
FROM Products
```

```
WHERE price > ALL (SELECT price FROM Products WHERE stock < 50);
```

(... continue up to **Q80** with ANY/ALL)

◆ Nested & Advanced Subqueries (Q81–Q100)

Q81. Find the second highest salary.

```
SELECT MAX(salary) AS second_highest
```

```
FROM Employees
```

```
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

Q82. Find the third highest salary.

```
SELECT MIN(salary) AS third_highest
```

```
FROM (SELECT DISTINCT TOP 3 salary FROM Employees ORDER BY salary DESC) t;
```

Q83. Find employees earning top 3 salaries.

```
SELECT * FROM Employees  
  
WHERE salary IN (SELECT DISTINCT TOP 3 salary FROM Employees ORDER BY salary  
DESC);
```

Q84. Find employees with orders greater than company avg order.

```
SELECT first_name, last_name  
  
FROM Employees  
  
WHERE employee_id IN (SELECT employee_id FROM Orders  
  
GROUP BY employee_id  
  
HAVING SUM(amount) > (SELECT AVG(amount) FROM Orders));
```

Q85. Find departments where no employees exist.

```
SELECT department_name  
  
FROM Departments  
  
WHERE department_id NOT IN (SELECT DISTINCT department_id FROM Employees  
WHERE department_id IS NOT NULL);
```

Q86. Find customers who never placed an order.

```
SELECT customer_name  
  
FROM Customers  
  
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM Orders);
```

Q87. Find employees earning same salary as someone else.

```
SELECT first_name, last_name, salary  
  
FROM Employees e  
  
WHERE salary IN (SELECT salary FROM Employees WHERE employee_id <>  
e.employee_id);
```

Q88. Find employees with salary above overall company average (nested).

```
SELECT first_name, last_name, salary  
  
FROM Employees
```

```
WHERE salary > (SELECT AVG(avg_salary) FROM (SELECT AVG(salary) AS avg_salary
                                                FROM Employees GROUP BY department_id) t);
```

Q89. Find customers who ordered all products.

```
SELECT customer_id
FROM Orders o
WHERE NOT EXISTS (SELECT product_id FROM Products p
                  WHERE NOT EXISTS (SELECT 1 FROM Orders o2
                                    WHERE o2.customer_id = o.customer_id
                                    AND o2.order_id = p.product_id));
```

Q90. Find employees hired after someone in same department.

```
SELECT first_name, last_name, hire_date, department_id
FROM Employees e1
WHERE hire_date > (SELECT MIN(hire_date) FROM Employees e2
                  WHERE e2.department_id = e1.department_id);
```

Q91. Find customers whose total order amount > average customer total.

```
SELECT customer_id
FROM Orders o
GROUP BY customer_id
HAVING SUM(amount) > (SELECT AVG(total_amt)
                     FROM (SELECT SUM(amount) AS total_amt
                           FROM Orders GROUP BY customer_id) t);
```

Q92. Find the employee with the earliest hire_date.

```
SELECT * FROM Employees
WHERE hire_date = (SELECT MIN(hire_date) FROM Employees);
```

Q93. Find employees earning below company median salary.

```
SELECT * FROM Employees
```

```
WHERE salary < (SELECT AVG(salary)
                FROM (SELECT TOP 50 PERCENT salary
                      FROM Employees ORDER BY salary) t);
```

Q94. Find the highest paid employee(s) in each department (subquery with IN).

```
SELECT * FROM Employees e
WHERE salary = (SELECT MAX(salary) FROM Employees WHERE department_id =
e.department_id);
```

Q95. Find employees who earn more than average salary of employees hired in 2020.

```
SELECT first_name, last_name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees WHERE YEAR(hire_date) = 2020);
```

Q96. Find employees whose salary is same as their manager.

```
SELECT e.first_name, e.last_name, e.salary
FROM Employees e
WHERE e.salary = (SELECT salary FROM Employees m WHERE m.employee_id =
e.manager_id);
```

Q97. Find products more expensive than avg price of ordered products.

```
SELECT product_name, price
FROM Products
WHERE price > (SELECT AVG(price) FROM Products WHERE product_id IN (SELECT
DISTINCT order_id FROM Orders));
```

Q98. Find employees in department with lowest avg salary.

```
SELECT * FROM Employees
WHERE department_id = (SELECT TOP 1 department_id FROM Employees GROUP BY
department_id ORDER BY AVG(salary));
```

Q99. Find employees whose salary rank is in top 10 (subquery).

```
SELECT * FROM Employees
```

WHERE salary IN (SELECT TOP 10 salary FROM Employees ORDER BY salary DESC);

Q100. Find customers whose max order amount is the overall max.

SELECT customer_id

FROM Orders

GROUP BY customer_id

HAVING MAX(amount) = (SELECT MAX(amount) FROM Orders);

String Functions (LEN, LEFT, RIGHT, SUBSTRING, UPPER, LOWER, LTRIM, RTRIM, CHARINDEX, REPLACE, CONCAT)

- **Date/Time Functions** (GETDATE, YEAR, MONTH, DAY, DATEADD, DATEDIFF, DATENAME)
 - **Mathematical Functions** (ABS, ROUND, CEILING, FLOOR, POWER, SQRT, RAND)
 - **System & Conversion Functions** (CAST, CONVERT, ISNULL, COALESCE, NULLIF)
-

SQL Functions Practice (100 Questions & Answers)

◆ String Functions (Q1–Q25)

Q1. Get the length of each employee's first name.

SELECT first_name, LEN(first_name) AS NameLength

FROM Employees;

Q2. Display first 3 characters of each last name.

SELECT last_name, LEFT(last_name, 3) AS ShortName

FROM Employees;

Q3. Display last 2 characters of employee IDs.

SELECT employee_id, RIGHT(employee_id, 2) AS LastTwoDigits

FROM Employees;

Q4. Extract 5 characters from first_name starting at position 2.

```
SELECT first_name, SUBSTRING(first_name, 2, 5) AS Extracted  
FROM Employees;
```

Q5. Convert employee names to uppercase.

```
SELECT UPPER(first_name) AS UpperName  
FROM Employees;
```

Q6. Convert customer names to lowercase.

```
SELECT LOWER(customer_name) AS LowerName  
FROM Customers;
```

Q7. Trim spaces from left of department_name.

```
SELECT LTRIM(department_name) AS Trimmed  
FROM Departments;
```

Q8. Trim spaces from right of department_name.

```
SELECT RTRIM(department_name) AS Trimmed  
FROM Departments;
```

Q9. Find position of character 'a' in first_name.

```
SELECT first_name, CHARINDEX('a', first_name) AS Position  
FROM Employees;
```

Q10. Replace 'Manager' with 'Lead' in job_title.

```
SELECT REPLACE(job_title, 'Manager', 'Lead') AS UpdatedTitle  
FROM Employees;
```

(... continue string functions up to **Q25**)

◆ Date/Time Functions (Q26–Q50)

Q26. Get today's date.

```
SELECT GETDATE() AS CurrentDateTime;
```

Q27. Get only the year from hire_date.

```
SELECT first_name, YEAR(hire_date) AS HireYear  
FROM Employees;
```

Q28. Get the month from hire_date.

```
SELECT first_name, MONTH(hire_date) AS HireMonth  
FROM Employees;
```

Q29. Get the day from hire_date.

```
SELECT first_name, DAY(hire_date) AS HireDay  
FROM Employees;
```

Q30. Add 1 year to hire_date.

```
SELECT first_name, DATEADD(YEAR, 1, hire_date) AS NextYear  
FROM Employees;
```

Q31. Subtract 30 days from order_date.

```
SELECT order_id, DATEADD(DAY, -30, order_date) AS AdjustedDate  
FROM Orders;
```

Q32. Find difference in years between hire_date and today.

```
SELECT first_name, DATEDIFF(YEAR, hire_date, GETDATE()) AS YearsWorked  
FROM Employees;
```

Q33. Find number of days between order_date and today.

```
SELECT order_id, DATEDIFF(DAY, order_date, GETDATE()) AS DaysAgo  
FROM Orders;
```

Q34. Get month name from hire_date.

```
SELECT first_name, DATENAME(MONTH, hire_date) AS HireMonthName  
FROM Employees;
```

Q35. Get day name from order_date.

```
SELECT order_id, DATENAME(WEEKDAY, order_date) AS DayName
```

FROM Orders;

(... continue date functions up to **Q50**)

◆ **Mathematical Functions (Q51–Q75)**

Q51. Get absolute value of -100.

```
SELECT ABS(-100) AS AbsoluteValue;
```

Q52. Round salary to nearest thousand.

```
SELECT first_name, ROUND(salary, -3) AS RoundedSalary  
FROM Employees;
```

Q53. Get ceiling of product price.

```
SELECT product_name, CEILING(price) AS CeilPrice  
FROM Products;
```

Q54. Get floor of product price.

```
SELECT product_name, FLOOR(price) AS FloorPrice  
FROM Products;
```

Q55. Raise salary to power of 2.

```
SELECT first_name, POWER(salary, 2) AS SalarySquared  
FROM Employees;
```

Q56. Get square root of salary.

```
SELECT first_name, SQRT(salary) AS SalarySqrt  
FROM Employees;
```

Q57. Generate a random number between 0 and 1.

```
SELECT RAND() AS RandomNumber;
```

Q58. Round product price to 2 decimals.

```
SELECT product_name, ROUND(price, 2) AS RoundedPrice  
FROM Products;
```

Q59. Find modulo of salary by 1000.

```
SELECT first_name, salary % 1000 AS SalaryModulo  
FROM Employees;
```

Q60. Increase order amount by 10% using math expression.

```
SELECT order_id, amount * 1.10 AS IncreasedAmount  
FROM Orders;
```

(... continue math functions up to **Q75**)

◆ System & Conversion Functions (Q76–Q100)

Q76. Convert hire_date to varchar.

```
SELECT first_name, CONVERT(VARCHAR, hire_date, 103) AS HireDateStr  
FROM Employees;
```

Q77. Cast salary to integer.

```
SELECT first_name, CAST(salary AS INT) AS SalaryInt  
FROM Employees;
```

Q78. Replace NULL commission with 0 using ISNULL.

```
SELECT first_name, ISNULL(commission, 0) AS Commission  
FROM Employees;
```

Q79. Replace NULL manager_id with -1 using COALESCE.

```
SELECT first_name, COALESCE(manager_id, -1) AS ManagerID  
FROM Employees;
```

Q80. Return NULL if salary = 0 using NULLIF.

```
SELECT first_name, NULLIF(salary, 0) AS SalaryCheck  
FROM Employees;
```

Q81. Convert string '2025-09-13' to DATE.

```
SELECT CAST('2025-09-13' AS DATE) AS ConvertedDate;
```

Q82. Convert float 123.45 to INT.

```
SELECT CAST(123.45 AS INT) AS IntegerValue;
```

Q83. Show employee details with salary converted to VARCHAR.

```
SELECT first_name, CAST(salary AS VARCHAR) AS SalaryStr  
FROM Employees;
```

Q84. Replace NULL department_id with text 'N/A'.

```
SELECT first_name, ISNULL(CAST(department_id AS VARCHAR), 'N/A') AS Dept  
FROM Employees;
```

Q85. Show order_date in YYYY-MM-DD format.

```
SELECT order_id, CONVERT(VARCHAR(10), order_date, 120) AS OrderDateFormatted  
FROM Orders;
```

(... continue conversion/system functions up to **Q100**)

This set will cover:

- **INNER JOIN**
 - **LEFT JOIN**
 - **RIGHT JOIN**
 - **FULL OUTER JOIN**
 - **CROSS JOIN**
 - **SELF JOIN**
-

 **SQL Joins Practice (100 Questions & Answers)**

INNER JOIN (Q1–Q20)

Q1. List employees with their department names.

```
SELECT e.first_name, e.last_name, d.department_name
```

FROM Employees e

INNER JOIN Departments d ON e.department_id = d.department_id;

Q2. Get all orders with customer names.

SELECT o.order_id, o.amount, c.customer_name

FROM Orders o

INNER JOIN Customers c ON o.customer_id = c.customer_id;

Q3. List employees and their managers.

SELECT e.first_name AS Employee, m.first_name AS Manager

FROM Employees e

INNER JOIN Employees m ON e.manager_id = m.employee_id;

Q4. Show orders along with product details.

SELECT o.order_id, p.product_name, o.amount

FROM Orders o

INNER JOIN Products p ON o.order_id = p.product_id;

(... continue INNER JOINS up to **Q20**)

◆ **LEFT JOIN (Q21–Q40)**

Q21. List all employees and their department names (even if no department).

SELECT e.first_name, d.department_name

FROM Employees e

LEFT JOIN Departments d ON e.department_id = d.department_id;

Q22. Show all customers and their orders (include customers with no orders).

SELECT c.customer_name, o.order_id, o.amount

FROM Customers c

LEFT JOIN Orders o ON c.customer_id = o.customer_id;

Q23. Show all departments with employees (include empty departments).

```
SELECT d.department_name, e.first_name  
FROM Departments d  
LEFT JOIN Employees e ON d.department_id = e.department_id;  
(... continue LEFT JOINS up to Q40)
```

◆ RIGHT JOIN (Q41–Q60)

Q41. List all employees with department names using RIGHT JOIN.

```
SELECT e.first_name, d.department_name  
FROM Employees e  
RIGHT JOIN Departments d ON e.department_id = d.department_id;
```

Q42. Show orders with all customers (customers first).

```
SELECT c.customer_name, o.order_id, o.amount  
FROM Orders o  
RIGHT JOIN Customers c ON o.customer_id = c.customer_id;
```

Q43. Show employees and their managers using RIGHT JOIN.

```
SELECT e.first_name AS Employee, m.first_name AS Manager  
FROM Employees e  
RIGHT JOIN Employees m ON e.manager_id = m.employee_id;  
(... continue RIGHT JOINS up to Q60)
```

◆ FULL OUTER JOIN (Q61–Q75)

Q61. Show all employees and departments (include unmatched).

```
SELECT e.first_name, d.department_name  
FROM Employees e  
FULL OUTER JOIN Departments d ON e.department_id = d.department_id;
```

Q62. Show all customers and their orders (include customers without orders and orders without customers).

```
SELECT c.customer_name, o.order_id  
  
FROM Customers c  
  
FULL OUTER JOIN Orders o ON c.customer_id = o.customer_id;
```

Q63. Show all products and orders.

```
SELECT p.product_name, o.order_id  
  
FROM Products p  
  
FULL OUTER JOIN Orders o ON p.product_id = o.order_id;  
  
(... continue FULL OUTER JOINS up to Q75)
```

◆ **CROSS JOIN (Q76–Q85)**

Q76. Get a combination of all employees and departments.

```
SELECT e.first_name, d.department_name  
  
FROM Employees e  
  
CROSS JOIN Departments d;
```

Q77. Show all possible pairs of customers and products.

```
SELECT c.customer_name, p.product_name  
  
FROM Customers c  
  
CROSS JOIN Products p;
```

Q78. Generate all employee-customer combinations.

```
SELECT e.first_name, c.customer_name  
  
FROM Employees e  
  
CROSS JOIN Customers c;  
  
(... continue CROSS JOINS up to Q85)
```

◆ **SELF JOIN (Q86–Q100)**

Q86. List employees and their managers.

```
SELECT e.first_name AS Employee, m.first_name AS Manager
FROM Employees e
JOIN Employees m ON e.manager_id = m.employee_id;
```

Q87. Find employees who share the same department.

```
SELECT e1.first_name AS Emp1, e2.first_name AS Emp2, e1.department_id
FROM Employees e1
JOIN Employees e2 ON e1.department_id = e2.department_id
WHERE e1.employee_id < e2.employee_id;
```

Q88. Find employees earning more than their colleagues in same department.

```
SELECT e1.first_name, e1.salary, e2.first_name AS Colleague, e2.salary AS
ColleagueSalary
FROM Employees e1
JOIN Employees e2 ON e1.department_id = e2.department_id
WHERE e1.salary > e2.salary;
```

Q89. Pair employees hired on the same date.

```
SELECT e1.first_name AS Emp1, e2.first_name AS Emp2, e1.hire_date
FROM Employees e1
JOIN Employees e2 ON e1.hire_date = e2.hire_date
WHERE e1.employee_id < e2.employee_id;
```

Q90. Show employees with same manager.

```
SELECT e1.first_name AS Emp1, e2.first_name AS Emp2, e1.manager_id
FROM Employees e1
JOIN Employees e2 ON e1.manager_id = e2.manager_id
WHERE e1.employee_id < e2.employee_id;
```

(... continue self-joins up to **Q100**)

We'll cover:

- **UNION**
 - **UNION ALL**
 - **INTERSECT**
 - **EXCEPT**
-

SQL Set Operations Practice (100 Questions & Answers)

UNION (Q1–Q25)

Q1. Get all employee first names and customer names in one list.

```
SELECT first_name AS Name FROM Employees
```

```
UNION
```

```
SELECT customer_name FROM Customers;
```

Q2. Get all department names and product names.

```
SELECT department_name AS Name FROM Departments
```

```
UNION
```

```
SELECT product_name FROM Products;
```

Q3. List all employee IDs and order IDs.

```
SELECT employee_id AS ID FROM Employees
```

```
UNION
```

```
SELECT order_id FROM Orders;
```

Q4. Combine employee job titles and department names.

```
SELECT job_title AS Title FROM Employees
```

```
UNION
```

```
SELECT department_name FROM Departments;
```

Q5. Get unique years from hire_date and order_date.

```
SELECT YEAR(hire_date) AS Year FROM Employees
```

```
UNION
```

```
SELECT YEAR(order_date) FROM Orders;
```

(... continue UNION examples up to **Q25**)

◆ UNION ALL (Q26–Q50)

Q26. Get all employee first names and customer names (with duplicates).

```
SELECT first_name AS Name FROM Employees
```

```
UNION ALL
```

```
SELECT customer_name FROM Customers;
```

Q27. Combine all product names and department names (allow duplicates).

```
SELECT product_name FROM Products
```

```
UNION ALL
```

```
SELECT department_name FROM Departments;
```

Q28. Show all employee IDs and order IDs (duplicates included).

```
SELECT employee_id AS ID FROM Employees
```

```
UNION ALL
```

```
SELECT order_id FROM Orders;
```

Q29. List all job titles and department names (with duplicates).

```
SELECT job_title FROM Employees
```

```
UNION ALL
```

```
SELECT department_name FROM Departments;
```

Q30. Get all years from hire_date and order_date (duplicates included).

```
SELECT YEAR(hire_date) AS Year FROM Employees
```

UNION ALL

SELECT YEAR(order_date) FROM Orders;

(... continue UNION ALL examples up to **Q50**)

◆ INTERSECT (Q51–Q75)

Q51. Find names that exist in both employees and customers.

SELECT first_name FROM Employees

INTERSECT

SELECT customer_name FROM Customers;

Q52. Find department IDs that exist in both Employees and Departments tables.

SELECT department_id FROM Employees

INTERSECT

SELECT department_id FROM Departments;

Q53. Find IDs that are both employee_id and order_id.

SELECT employee_id AS ID FROM Employees

INTERSECT

SELECT order_id FROM Orders;

Q54. Find names that are both job_title and department_name.

SELECT job_title FROM Employees

INTERSECT

SELECT department_name FROM Departments;

Q55. Find years that are both in hire_date and order_date.

SELECT YEAR(hire_date) FROM Employees

INTERSECT

SELECT YEAR(order_date) FROM Orders;

(... continue INTERSECT examples up to **Q75**)

◆ EXCEPT (Q76–Q100)

Q76. Find employee names that are not customer names.

```
SELECT first_name FROM Employees
```

```
EXCEPT
```

```
SELECT customer_name FROM Customers;
```

Q77. Find department IDs in Departments but not in Employees.

```
SELECT department_id FROM Departments
```

```
EXCEPT
```

```
SELECT department_id FROM Employees;
```

Q78. Find employee IDs that are not order IDs.

```
SELECT employee_id FROM Employees
```

```
EXCEPT
```

```
SELECT order_id FROM Orders;
```

Q79. Find job titles not matching any department names.

```
SELECT job_title FROM Employees
```

```
EXCEPT
```

```
SELECT department_name FROM Departments;
```

Q80. Find years of hire_date not present in order_date.

```
SELECT YEAR(hire_date) FROM Employees
```

```
EXCEPT
```

```
SELECT YEAR(order_date) FROM Orders;
```

(... continue EXCEPT examples up to **Q100**)

We'll cover:

- **Ranking functions:** ROW_NUMBER, RANK, DENSE_RANK, NTILE

- **Analytic aggregates:** SUM() OVER, AVG() OVER, COUNT() OVER, MIN/MAX() OVER
 - **Window navigation:** LAG, LEAD, FIRST_VALUE, LAST_VALUE
-

SQL Window Functions Practice (100 Questions & Answers)

◆ ROW_NUMBER (Q1–Q15)

Q1. Assign row numbers to employees ordered by salary.

```
SELECT employee_id, first_name, salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS RowNum  
FROM Employees;
```

Q2. Assign row numbers to orders by order_date.

```
SELECT order_id, order_date,  
       ROW_NUMBER() OVER (ORDER BY order_date) AS RowNum  
FROM Orders;
```

Q3. Get top 5 highest-paid employees using ROW_NUMBER.

```
WITH Ranked AS (  
    SELECT first_name, salary,  
           ROW_NUMBER() OVER (ORDER BY salary DESC) AS RowNum  
    FROM Employees  
)  
SELECT * FROM Ranked WHERE RowNum <= 5;  
(... continue ROW_NUMBER up to Q15)
```

◆ RANK (Q16–Q30)

Q16. Rank employees by salary.

```
SELECT first_name, salary,
```

```
RANK() OVER (ORDER BY salary DESC) AS SalaryRank  
FROM Employees;
```

Q17. Rank customers by total order amount.

```
SELECT customer_id, SUM(amount) AS TotalAmount,  
       RANK() OVER (ORDER BY SUM(amount) DESC) AS CustRank  
FROM Orders  
GROUP BY customer_id;
```

Q18. Rank products by price.

```
SELECT product_name, price,  
       RANK() OVER (ORDER BY price DESC) AS PriceRank  
FROM Products;  
(... continue RANK up to Q30)
```

◆ DENSE_RANK (Q31–Q45)

Q31. Dense rank employees by salary.

```
SELECT first_name, salary,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS DenseRank  
FROM Employees;
```

Q32. Dense rank orders by amount.

```
SELECT order_id, amount,  
       DENSE_RANK() OVER (ORDER BY amount DESC) AS OrderRank  
FROM Orders;
```

Q33. Find top 3 unique salaries.

```
WITH Ranked AS (  
    SELECT salary,  
           DENSE_RANK() OVER (ORDER BY salary DESC) AS RankNum
```

```
FROM Employees
)
SELECT * FROM Ranked WHERE RankNum <= 3;
(... continue DENSE_RANK up to Q45)
```

◆ NTILE (Q46–Q60)

Q46. Divide employees into 4 salary quartiles.

```
SELECT first_name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS Quartile
FROM Employees;
```

Q47. Divide customers into 5 groups based on total orders.

```
SELECT customer_id, SUM(amount) AS TotalAmount,
       NTILE(5) OVER (ORDER BY SUM(amount) DESC) AS GroupNum
FROM Orders
GROUP BY customer_id;
```

Q48. Split products into 3 groups by price.

```
SELECT product_name, price,
       NTILE(3) OVER (ORDER BY price DESC) AS PriceGroup
FROM Products;
(... continue NTILE up to Q60)
```

◆ Aggregates with OVER (Q61–Q80)

Q61. Show salary along with department average.

```
SELECT first_name, salary, department_id,
       AVG(salary) OVER (PARTITION BY department_id) AS DeptAvg
FROM Employees;
```

Q62. Show order amount with running total by customer.

```
SELECT order_id, customer_id, amount,  
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS  
       RunningTotal  
FROM Orders;
```

Q63. Show min and max salary in each department.

```
SELECT first_name, salary, department_id,  
       MIN(salary) OVER (PARTITION BY department_id) AS MinSalary,  
       MAX(salary) OVER (PARTITION BY department_id) AS MaxSalary  
FROM Employees;  
(... continue aggregates up to Q80)
```

◆ **LAG, LEAD, FIRST_VALUE, LAST_VALUE (Q81–Q100)**

Q81. Compare employee salary with previous employee's salary.

```
SELECT first_name, salary,  
       LAG(salary, 1) OVER (ORDER BY salary) AS PrevSalary  
FROM Employees;
```

Q82. Compare employee salary with next employee's salary.

```
SELECT first_name, salary,  
       LEAD(salary, 1) OVER (ORDER BY salary) AS NextSalary  
FROM Employees;
```

Q83. Get first order amount for each customer.

```
SELECT customer_id, order_id, amount,  
       FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS  
       FirstOrder  
FROM Orders;
```

Q84. Get last order amount for each customer.

```
SELECT customer_id, order_id, amount,  
       LAST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_date  
                               ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED  
FOLLOWING) AS LastOrder  
FROM Orders;
```

Q85. Find salary difference with previous employee in same department.

```
SELECT first_name, department_id, salary,  
       salary - LAG(salary) OVER (PARTITION BY department_id ORDER BY salary) AS  
DiffWithPrev  
FROM Employees;  
(... continue up to Q100)
```

We'll cover:

- **CTEs (Common Table Expressions)** – Q1–Q25
 - **Recursive CTEs** – Q26–Q45
 - **Pivot & Unpivot** – Q46–Q65
 - **Advanced Subqueries (ANY, ALL, EXISTS, correlated)** – Q66–Q85
 - **Other Advanced SQL Features (CTE + Window mix, derived tables, case statements, etc.)** – Q86–Q100
-

 SQL Advanced Topics Practice (100 Questions & Answers)

 CTEs (Q1–Q25)

Q1. Show employees with salary above department average using CTE.

```
WITH DeptAvg AS (  
    SELECT department_id, AVG(salary) AS AvgSalary
```

```
FROM Employees
GROUP BY department_id
)
SELECT e.first_name, e.salary, e.department_id
FROM Employees e
JOIN DeptAvg d ON e.department_id = d.department_id
WHERE e.salary > d.AvgSalary;
```

Q2. Get top 3 highest paid employees using CTE.

```
WITH Ranked AS (
    SELECT first_name, salary,
           ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
    FROM Employees
)
SELECT * FROM Ranked WHERE rn <= 3;
(... continue until Q25 with filtering, grouping, joins inside CTEs)
```

◆ Recursive CTEs (Q26–Q45)

Q26. Display numbers from 1 to 10 using recursive CTE.

```
WITH Numbers AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM Numbers WHERE n < 10
)
SELECT * FROM Numbers;
```

Q27. Show employee hierarchy (manager → employee).

```
WITH EmpHierarchy AS (
```

```
SELECT employee_id, first_name, manager_id, 1 AS level
FROM Employees WHERE manager_id IS NULL

UNION ALL

SELECT e.employee_id, e.first_name, e.manager_id, h.level + 1
FROM Employees e
JOIN EmpHierarchy h ON e.manager_id = h.employee_id
)

SELECT * FROM EmpHierarchy;

(... continue until Q45)
```

◆ Pivot & Unpivot (Q46–Q65)

Q46. Pivot total sales per customer by year.

```
SELECT customer_id, [2023] AS Sales2023, [2024] AS Sales2024
FROM (
    SELECT customer_id, YEAR(order_date) AS OrderYear, amount
    FROM Orders
) src
PIVOT (
    SUM(amount) FOR OrderYear IN ([2023], [2024])
) p;
```

Q47. Pivot count of employees by department.

```
SELECT *
FROM (
    SELECT department_id, gender
    FROM Employees
) src
```

```
PIVOT (  
    COUNT(gender) FOR gender IN ([Male], [Female])  
) p;
```

Q48. Unpivot sales data from columns to rows.

```
SELECT customer_id, OrderYear, Amount  
FROM SalesByYear  
UNPIVOT (  
    Amount FOR OrderYear IN ([2023], [2024], [2025])  
) u;  
(... continue until Q65)
```

◆ **Advanced Subqueries (ANY, ALL, EXISTS) (Q66–Q85)**

Q66. Find employees who earn more than *any* employee in department 2.

```
SELECT *  
FROM Employees  
WHERE salary > ANY (SELECT salary FROM Employees WHERE department_id = 2);
```

Q67. Find employees who earn more than *all* employees in department 2.

```
SELECT *  
FROM Employees  
WHERE salary > ALL (SELECT salary FROM Employees WHERE department_id = 2);
```

Q68. Find customers who have placed at least one order (EXISTS).

```
SELECT *  
FROM Customers c  
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id);
```

Q69. Find products not ordered by any customer (NOT EXISTS).

```
SELECT *
```

FROM Products p

WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.product_id = p.product_id);

(... continue until **Q85**)

◆ Other Advanced SQL Features (Q86–Q100)

Q86. Use CASE to classify employees by salary range.

SELECT first_name, salary,

CASE

WHEN salary < 3000 THEN 'Low'

WHEN salary BETWEEN 3000 AND 7000 THEN 'Medium'

ELSE 'High'

END AS SalaryCategory

FROM Employees;

Q87. Use CTE with window function to get top 2 earners per department.

WITH Ranked AS (

SELECT first_name, department_id, salary,

ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn

FROM Employees

)

SELECT * FROM Ranked WHERE rn <= 2;

Q88. Show moving average of salaries.

SELECT employee_id, salary,

AVG(salary) OVER (ORDER BY employee_id ROWS BETWEEN 2 PRECEDING AND
CURRENT ROW) AS MovingAvg

FROM Employees;

Q89. Derived table example – top 5 products by sales.

```
SELECT product_id, SUM(amount) AS TotalSales
FROM Orders
GROUP BY product_id
ORDER BY TotalSales DESC
OFFSET 0 ROWS FETCH NEXT 5 ROWS ONLY;
```

Q90. Find duplicate customer emails.

```
SELECT email, COUNT(*) AS Cnt
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1;
(... continue until Q100)
```

We'll cover:

- **Indexes** – Q1–Q20
 - **Transactions & Isolation Levels** – Q21–Q40
 - **Error Handling (TRY...CATCH, RAISERROR)** – Q41–Q60
 - **Triggers** – Q61–Q80
 - **Stored Procedures & Functions** – Q81–Q100
-

 SQL Server Administration & Performance Tuning Practice (100 Q&A)

◆ **Indexes (Q1–Q20)**

Q1. Create a clustered index on Employees' salary.

```
CREATE CLUSTERED INDEX IDX_Employees_Salary
ON Employees(salary);
```

Q2. Create a non-clustered index on Customers' email.

```
CREATE NONCLUSTERED INDEX IDX_Customers_Email  
ON Customers(email);
```

Q3. Drop an index on Orders table.

```
DROP INDEX IDX_Orders_Amount ON Orders;
```

Q4. Include columns in non-clustered index.

```
CREATE NONCLUSTERED INDEX IDX_Orders_CustomerAmount  
ON Orders(customer_id)  
INCLUDE(amount, order_date);
```

Q5. Check existing indexes on Employees table.

```
EXEC sp_helpindex 'Employees';  
(... continue index examples up to Q20)
```

◆ Transactions & Isolation Levels (Q21–Q40)

Q21. Start a transaction and commit.

```
BEGIN TRAN;  
UPDATE Employees SET salary = salary + 1000 WHERE department_id = 10;  
COMMIT TRAN;
```

Q22. Start a transaction and rollback.

```
BEGIN TRAN;  
UPDATE Employees SET salary = salary + 1000 WHERE department_id = 10;  
ROLLBACK TRAN;
```

Q23. Set transaction isolation level to READ COMMITTED.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Q24. Set transaction isolation level to SERIALIZABLE.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Q25. Demonstrate nested transactions.

```
BEGIN TRAN OuterTran;

    UPDATE Employees SET salary = salary + 500;

    BEGIN TRAN InnerTran;

        UPDATE Employees SET salary = salary + 200;

    COMMIT TRAN InnerTran;

COMMIT TRAN OuterTran;

(... continue transactions up to Q40)
```

◆ **Error Handling (TRY...CATCH, RAISERROR) (Q41–Q60)**

Q41. Basic TRY...CATCH block.

```
BEGIN TRY

    UPDATE Employees SET salary = salary / 0;

END TRY

BEGIN CATCH

    SELECT ERROR_MESSAGE() AS ErrorMessage;

END CATCH;
```

Q42. Raise a custom error.

```
RAISERROR('Salary cannot be negative', 16, 1);
```

Q43. Use TRY...CATCH with transaction.

```
BEGIN TRAN;

BEGIN TRY

    UPDATE Employees SET salary = -100 WHERE employee_id = 1;

    COMMIT TRAN;

END TRY

BEGIN CATCH

    ROLLBACK TRAN;
```

```
SELECT ERROR_MESSAGE() AS ErrorMessage;  
END CATCH;
```

Q44. Check XACT_STATE() in CATCH block.

```
BEGIN CATCH  
    IF XACT_STATE() <> 0  
        ROLLBACK TRAN;  
        SELECT ERROR_MESSAGE() AS ErrorMessage;  
END CATCH;  
(... continue error handling up to Q60)
```

◆ **Triggers (Q61–Q80)**

Q61. Create AFTER INSERT trigger on Employees.

```
CREATE TRIGGER trg_AfterInsert_Employees  
ON Employees  
AFTER INSERT  
AS  
BEGIN  
    PRINT 'New employee inserted';  
END;
```

Q62. Create AFTER UPDATE trigger to log salary changes.

```
CREATE TRIGGER trg_AfterUpdate_Salary  
ON Employees  
AFTER UPDATE  
AS  
BEGIN  
    INSERT INTO SalaryAudit(employee_id, old_salary, new_salary, change_date)
```

```
SELECT i.employee_id, d.salary, i.salary, GETDATE()
FROM inserted i
JOIN deleted d ON i.employee_id = d.employee_id;
END;
```

Q63. Drop a trigger.

```
DROP TRIGGER trg_AfterInsert_Employees;
```

Q64. Create INSTEAD OF DELETE trigger on Departments.

```
CREATE TRIGGER trg_InsteadDelete_Departments
ON Departments
INSTEAD OF DELETE
AS
BEGIN
    PRINT 'Departments cannot be deleted';
END;
(... continue triggers up to Q80)
```

◆ Stored Procedures & Functions (Q81–Q100)

Q81. Create a simple stored procedure to get employee by ID.

```
CREATE PROCEDURE sp_GetEmployeeByID
    @EmpID INT
AS
BEGIN
    SELECT * FROM Employees WHERE employee_id = @EmpID;
END;
```

Q82. Execute the stored procedure.

```
EXEC sp_GetEmployeeByID @EmpID = 1;
```

Q83. Create stored procedure with output parameter.

```
CREATE PROCEDURE sp_GetSalary
    @EmpID INT,
    @Salary MONEY OUTPUT
AS
BEGIN
    SELECT @Salary = salary FROM Employees WHERE employee_id = @EmpID;
END;
```

Q84. Call procedure with output parameter.

```
DECLARE @Sal MONEY;
EXEC sp_GetSalary @EmpID = 1, @Salary = @Sal OUTPUT;
SELECT @Sal AS Salary;
```

Q85. Create scalar function to calculate yearly bonus.

```
CREATE FUNCTION fn_CalcBonus(@Salary MONEY)
RETURNS MONEY
AS
BEGIN
    RETURN @Salary * 0.10;
END;
```

Q86. Use function in SELECT.

```
SELECT first_name, salary, dbo.fn_CalcBonus(salary) AS Bonus
FROM Employees;
```

Q87. Create table-valued function for employees in a department.

```
CREATE FUNCTION fn_GetEmployeesByDept(@DeptID INT)
RETURNS TABLE
AS
```

RETURN

SELECT * FROM Employees WHERE department_id = @DeptID;

Q88. Call table-valued function.

SELECT * FROM dbo.fn_GetEmployeesByDept(10);

Q89. Stored procedure with TRY...CATCH inside.

CREATE PROCEDURE sp_UpdateSalary

@EmpID INT, @Inc MONEY

AS

BEGIN

BEGIN TRY

UPDATE Employees SET salary = salary + @Inc WHERE employee_id = @EmpID;

END TRY

BEGIN CATCH

SELECT ERROR_MESSAGE() AS ErrorMessage;

END CATCH

END;

Q90. Execute stored procedure with error handling.

EXEC sp_UpdateSalary @EmpID = 1, @Inc = -5000;

(... continue stored procedures & functions up to **Q100**)
